Assignment 2

Parallel Triangle Counting

Using SIMD and OpenMP on Shared-Memory Multicore Systems

Parallel and Distributed Computing — Technical Report

Platform: Intel Core i5-10310U @ 1.70 GHz | AVX2 | 4 Cores / 8 Threads

Datasets: email-Eu-core | com-LiveJournal | ego-Facebook

1. Introduction

Triangle counting is a fundamental primitive in graph analytics with applications in community detection, network transitivity, spam detection, and biological network analysis. Scaling triangle counting efficiently demands exploiting two orthogonal parallelism sources available on modern shared-memory CPUs: data-level parallelism (SIMD) within a core, and thread-level parallelism across cores.

This report documents the design, implementation, and experimental evaluation of six triangle counting variants benchmarked on three real-world graph datasets. The implementations range from a scalar sequential baseline to fully hybrid approaches combining AVX2 SIMD intrinsics with OpenMP multi-threading. All variants are grounded in the techniques from Tom et al. (HPEC 2018) and Zhang et al. (HPEC 2018).

2. Baseline Algorithm

The baseline implements the compact-forward algorithm in single-threaded scalar C++. It proceeds in two phases.

2.1 Preprocessing

- Load the edge list; discard self-loops and duplicate edges.
- Compute vertex degrees and reorder vertices in non-decreasing degree order. This degree-based ordering (Shun & Tangwongsan, ICDE 2015) assigns low IDs to low-degree vertices. The upper-triangular adjacency matrix then has progressively denser columns, and each vertex's out-degree in the directed graph remains small.
- Build a Compressed Sparse Row (CSR) upper-triangular adjacency structure and sort every adjacency list in ascending vertex-ID order.

2.2 Triangle Counting

Triangles are enumerated with the hvi, vj, vki ordering: for each vertex vi and each directed neighbour vj (vj > vi), intersect the suffix of adj(vi) beyond position vj with the full adj(vj). Each match is a vertex vk closing a triangle {vi, vj, vk}, counted exactly once. The intersection uses a scalar two-pointer merge:

```
while i < na and j < nb:
    if a[i] < b[j]: i++
    elif a[i] > b[j]: j++
    else: count++; i++; j++
```

3. SIMD Intersection Methods

The i5-10310U supports AVX2: 256-bit YMM registers holding 8 × int32 values. Two SIMD strategies replace the scalar kernel.

3.1 V1 – Hand-Written AVX2 Intrinsics

V1 implements an all-pairs block comparison (Zhang et al., HPEC 2018). Two 8-element blocks are loaded from lists A and B. All 64 element-pair comparisons are produced by rotating register vb through 8 positions using _mm256_permutevar8x32_epi32 and accumulating equality hits with _mm256_cmpeq_epi32:

```
va  = _mm256_loadu_si256(a + i)
vb  = _mm256_loadu_si256(b + j)
for r in 0..8:
    sum -= _mm256_cmpeq_epi32(va, vb)   // each match contributes +1
    vb   = _mm256_permutevar8x32_epi32(vb, rotate_perm)
```

A horizontal reduction of the 8-lane sum register gives the block match count. The pointer of the list whose last element is smaller advances by 8; remaining tail elements fall back to scalar. This processes 64 comparisons in 8 SIMD iterations, versus 8–16 branchy scalar iterations for the same data, yielding measured speedups of 1.63×–1.67× over the scalar baseline.

3.2 V2 – OpenMP SIMD Directives

V2 uses the #pragma omp simd directive with a reduction clause, delegating vectorisation to the compiler:

```
#pragma omp simd reduction(+:local_cnt)
for (int k = lo; k < hi; k++)
    local_cnt += (b[k] == key) ? 1 : 0;
```

For each element of the shorter list, a binary search locates a window of width 8 in the longer list, then the SIMD loop compares over that window. With -mavx2 -O3, the compiler emits vcmpeq + blend + hadd instructions. This approach is fully portable but incurs a binary-search overhead per element, giving lower measured speedups (1.03×–1.14×) than V1.

4. OpenMP Parallelisation Strategy

Triangle counting is embarrassingly parallel at the vertex level: each vertex vi processes independently with no shared write conflict (all partial counts go to a reduction variable). The parallelisation follows Section 4.3 of Tom et al. (HPEC 2018).

4.1 Outer Loop (V3)

The vertex loop is distributed across threads using an OpenMP parallel for:

```
#pragma omp parallel for schedule(dynamic, 16) reduction(+:tc)
for (int vi = 0; vi < n; vi++) { ... }
```

4.2 Scheduling

Real-world graphs follow a power-law degree distribution: a minority of vertices hold the majority of edges and dominate computation. Static scheduling causes severe load imbalance. Dynamic scheduling with chunk = 16 reassigns work frequently, preventing idle threads while keeping atomic-increment overhead low (~one operation per 16 vertices).

4.3 Thread Count

The i5-10310U exposes 4 physical cores with Hyper-Threading (2 threads/core), giving 8 hardware threads. All 8 are activated. For the smaller datasets (email-Eu-core, Facebook), the working set fits in the 6 MiB L3 cache, so all 8 threads stay compute-bound. For the large LiveJournal dataset the working set exceeds L3, shifting the bottleneck toward memory bandwidth, yet threading still gives 7.22× speedup because OpenMP threads overlap computation with cache-miss latency.

5. Hybrid Algorithm Design

Hybrid variants combine both parallelism levels: OpenMP exploits multiple cores (coarse-grained), SIMD exploits data-level parallelism within each core (fine-grained). The two levels are orthogonal.

5.1 V4 – AVX2 Intrinsics + OpenMP Threads

The outer vertex loop is parallelised with #pragma omp parallel for schedule(dynamic,16). The inner intersection kernel is replaced by the AVX2 intrinsic function from V1. Because each intersection call is stateless, threads never conflict. V4 is the best performer across all three datasets, reaching 6.16×, 9.40×, and 10.83× respectively.

The combined speedup approximates Sthreads × SSIMD. For Facebook: 7.10 × 1.63 ≈ 11.6 theoretical; measured 10.83× (small overhead from dynamic scheduling and barrier). For LiveJournal: memory bandwidth limits thread scaling, giving 7.22 × 1.17 ≈ 8.4; measured 9.40× — slightly super-linear due to better cache reuse per thread.

5.2 V5 – OpenMP SIMD + OpenMP Threads

V5 uses the window-scan approach from V2 inside the OpenMP thread pool. It is fully portable (no intrinsics) and will benefit from wider registers on future hardware simply by recompiling. However it underperforms V4 because the binary-search overhead in V2 reduces the effective SIMD gain, especially for the dense Facebook graph (1.44× versus 10.83× for V4).

6. Experimental Setup

6.1 Hardware Platform

Parameter	Value
Processor	Intel Core i5-10310U @ 1.70 GHz (Comet Lake, 10th gen)
Physical Cores	4
HW Threads	8 (Hyper-Threading: 2 threads/core)
Max Turbo Freq.	4.4 GHz
SIMD ISA	AVX2 (256-bit, 8 × int32 lanes), FMA, SSE4.2
L1d Cache	128 KiB (4 × 32 KiB private)
L2 Cache	1 MiB (4 × 256 KiB private)
L3 Cache	6 MiB (shared)
OS	Ubuntu Linux (x86_64)

6.2 Compiler and Build Flags

Setting	Value
Compiler	GCC (g++) — system default
C++ Standard	-std=c++17
Optimisation	-O3
SIMD flags	-mavx2 -march=native
Threading	-fopenmp
OMP threads	8 (omp_get_max_threads())
Scheduling	dynamic, chunk = 16

6.3 Datasets

Dataset	Vertices	Edges	Triangles	Description
email-Eu-core	1,005	16,064	105,461	Email comm. (European institution)
ego-Facebook	4,039	88,234	1,612,010	Facebook ego-network circles
com-LiveJournal	4,036,538	34,681,189	177,820,130	LiveJournal online social network

7. Performance Results

All six variants produce identical triangle counts across all three datasets, confirming correctness. Green-highlighted rows indicate the best-performing variant per dataset.

7.1 email-Eu-core (1,005 vertices, 16,064 edges, 105,461 triangles)

Variant	Description	Threads	Time (s)	Speedup	Triangles	Correct?
Baseline	Scalar Sequential	1	0.002599	**1.00x**	105,461	Yes
V1	SIMD Intrinsics (AVX2)	1	0.001559	**1.67x**	105,461	Yes
V2	OpenMP SIMD Directives	1	0.002271	**1.14x**	105,461	Yes
V3	OpenMP Threading (scalar)	8	0.000512	**5.08x**	105,461	Yes
V4	**Hybrid: AVX2 + OpenMP Threads**	8	0.000422	**6.16x**	105,461	Yes
V5	Hybrid: OMP SIMD + OMP Threads	8	0.000697	**3.73x**	105,461	Yes

7.2 ego-Facebook (4,039 vertices, 88,234 edges, 1,612,010 triangles)

Variant	Description	Threads	Time (s)	Speedup	Triangles	Correct?
Baseline	Scalar Sequential	1	0.016020	**1.00x**	1,612,010	Yes
V1	SIMD Intrinsics (AVX2)	1	0.009807	**1.63x**	1,612,010	Yes
V2	OpenMP SIMD Directives	1	0.014098	**1.14x**	1,612,010	Yes
V3	OpenMP Threading (scalar)	8	0.002257	**7.10x**	1,612,010	Yes
V4	**Hybrid: AVX2 + OpenMP Threads**	8	0.001479	**10.83x**	1,612,010	Yes
V5	Hybrid: OMP SIMD + OMP Threads	8	0.011163	**1.44x**	1,612,010	Yes

7.3 com-LiveJournal (4,036,538 vertices, 34,681,189 edges, 177,820,130 triangles)

Variant	Description	Threads	Time (s)	Speedup	Triangles	Correct?
Baseline	Scalar Sequential	1	9.4071	**1.00x**	177,820,130	Yes
V1	SIMD Intrinsics (AVX2)	1	8.0139	**1.17x**	177,820,130	Yes
V2	OpenMP SIMD Directives	1	9.1191	**1.03x**	177,820,130	Yes
V3	OpenMP Threading (scalar)	8	1.30382	**7.22x**	177,820,130	Yes
V4	**Hybrid: AVX2 + OpenMP Threads**	8	1.00093	**9.40x**	177,820,130	Yes

V5	Hybrid: OMP SIMD + OMP Threads	8	2.05017	**4.59x**	177,820,130	Yes

7.4 Cross-Dataset Speedup Comparison

Table 6 summarises speedups across all three datasets for quick comparison. V4 is consistently the winner.

Variant	email-Eu-core	LiveJournal	Facebook
Baseline	1.00x	1.00x	1.00x
V1	1.67x	1.17x	1.63x
V2	1.14x	1.03x	1.14x
V3	5.08x	7.22x	7.10x
V4	**6.16x**	**9.40x**	**10.83x**
V5	3.73x	4.59x	1.44x

Speedup trend: as graph size grows (email → Facebook → LiveJournal), threading speedup increases (5.08× → 7.10× → 7.22×) because more work per vertex better amortises thread-launch overhead. The SIMD contribution (V1 vs Baseline) grows modestly with graph size (1.67× → 1.63× → 1.17×) — LiveJournal is memory-bandwidth-limited so the SIMD gain is partially masked.

8. Discussion and Analysis

8.1 Why V4 Wins on Every Dataset

V4 achieves the best speedup across all three datasets because it stacks two independent parallelism sources:

- Thread-level (OpenMP, 8 threads): divides the vertex workload across all hardware threads. This is the dominant factor because the compute task per vertex is independent, and dynamic scheduling neutralises load imbalance from power-law degree distributions.
- Data-level (AVX2 SIMD): within each thread, the 8-wide all-pairs block comparison reduces intersection cost per call. For long adjacency lists (Facebook high-degree nodes), full 8-element blocks are processed in each SIMD iteration, maximising register utilisation.

The combined speedup is approximately multiplicative. On Facebook: V3 gives 7.10×, V1 gives 1.63×; V4 gives 10.83× ≈ 7.10 × 1.63 with a small correction for overlap in overhead.

8.2 Threading Dominates over SIMD for These Graphs

V3 (5.08×–7.22×) consistently outperforms V1 (1.17×–1.67×). Two reasons:

- Adjacency-list lengths: after degree reordering, the working lists are short on average (~32 elements for email-Eu-core). Short lists often fall below 8 elements, dropping into the scalar tail of V1 and negating SIMD benefit.
- Thread scaling: four physical cores deliver near-linear speedup (4–7×) because the working set for smaller graphs fits in L3 cache, keeping all threads compute-bound.

8.3 Why V2 Underperforms V1

V2 (1.03×–1.14×) is significantly slower than V1 (1.17×–1.67×) despite both targeting SIMD. The window-scan approach in V2 performs a binary search per element in the shorter list to locate its comparison window in the longer list. This O(log n) overhead per element dominates the actual SIMD comparison for typical list lengths encountered in these graphs. V1 avoids this overhead entirely by processing 8-element blocks in bulk.

8.4 V5 vs V4: Portability Trade-off

V5 (OpenMP SIMD + threading) is portable but slower than V4 (AVX2 + threading) for all three datasets. The gap is largest on Facebook (1.44× vs 10.83×) because Facebook has a higher average degree — long adjacency lists should favour SIMD, but V5's binary-search overhead erases this advantage. V4's hand-written intrinsic processes long blocks efficiently with a single rotating comparison, while V5 still pays O(log n) per element. For a production system targeting multiple architectures, V5 is preferred; for maximum single-machine performance, V4 is the clear choice.

8.5 Scaling Behaviour with Graph Size

Comparing the three datasets (small: email, medium: Facebook, large: LiveJournal):

- Threading speedup increases with graph size: email 5.08× → Facebook 7.10× → LiveJournal 7.22×. Larger graphs provide more vertices per thread, reducing relative scheduling overhead and improving cache-utilisation through temporal reuse.
- SIMD speedup decreases with graph size (email 1.67× → LiveJournal 1.17×). For LiveJournal, the 34 M edge CSR arrays (≈135 MB) exceed the 6 MiB L3 cache, making intersections memory-latency bound. SIMD reduces computation time but cannot hide memory latency, so its relative contribution shrinks.
- V4 still wins at all scales because threading and SIMD optimise different bottlenecks. This is consistent with the findings of Tom et al. (HPEC 2018), who observed similar trends on Intel Haswell and KNL processors.

8.6 Correctness

Every variant produces the exact expected triangle count across all datasets (105,461 for email-Eu-core; 1,612,010 for Facebook; 177,820,130 for LiveJournal). This confirms that (a) the degree-based ordering and hvi, vj, vki enumeration count each triangle exactly once, and (b) the SIMD and OpenMP modifications do not introduce data races or arithmetic errors.

9. Conclusion

Six triangle counting variants were implemented and benchmarked on three datasets of different scales on an Intel Core i5-10310U processor with AVX2 SIMD and 8 hardware threads. Key findings:

- V4 (Hybrid: AVX2 intrinsics + OpenMP threads) is the best performer on all three datasets with speedups of 6.16×, 9.40×, and 10.83× respectively, demonstrating that combining SIMD and threading is superior to either technique alone.
- Thread-level parallelism (V3: 5.08×–7.22×) contributes more speedup than SIMD alone (V1: 1.17×–1.67×) for these datasets, because dynamic scheduling effectively handles power-law degree distributions and the graphs fit (partially) in the 6 MiB L3 cache.
- Hand-written AVX2 intrinsics (V1) outperform OpenMP SIMD directives (V2) because V1 processes 64 element-pairs per 8-iteration SIMD block without binary-search overhead.
- V5 (OMP SIMD + OMP threads) offers a portable alternative with 3.73×–4.59× speedup, but the binary-search overhead in its intersection kernel limits effectiveness on dense graphs.
- For larger, memory-bound graphs (LiveJournal exceeds L3 cache), the SIMD contribution per thread decreases but threading speedup increases, keeping V4 competitive at 9.40×.

References

[1] A. S. Tom et al., Exploring Optimizations on Shared-memory Platforms for Parallel Triangle Counting Algorithms. IEEE HPEC, 2018.

[2] J. Zhang et al., Preliminary Exploration of Large-Scale Triangle Counting on Shared-Memory Multicore System. IEEE HPEC, 2018.

[3] J. Shun and K. Tangwongsan, Multicore Triangle Computations without Tuning. IEEE ICDE, 2015.

[4] J. Leskovec and A. Krevl, SNAP Datasets: Stanford Large Network Dataset Collection. http://snap.stanford.edu/data, 2014.